

Formal Languages and Compilers
Prof. Breveglieri, Morzenti, Agosta
Written exam: laboratory question

05/07/2022

Time: 60 minutes. Textbooks and notes can be used. Pencil writing is allowed.

Important: Write your name on any additional sheet.

SURNAME (Cognome):

NAME (Nome):

Matricola:or Person Code:

Instructor: ☐ Prof. Breviglieri ☐ Prof. Morzenti ☐ Prof. Agosta

The laboratory question must be answered taking into account the implementation of the Acse compiler given with the exam text.

Modify the specification of the lexical analyser (`flex` input) and the syntactic analyser (`bison` input) and any other source file required to extend the Lance language with the ability to count how many bits set to 1 are present in the binary representation of an integer value.

This task is performed by a new operator, called **count_ones**, which takes an **arbitrary expression** as its only argument. The value of the operator is the number of bits set to 1 in the binary representation of the value of the expression. The implementation of the operator must perform constant folding whenever possible and must handle negative numbers correctly considering their two's complement representation. Due to the fact that the Lance language only supports 32-bit sized integers, the maximum result of `count_ones` must be 32.

The operation and syntax of the `count_ones` operator is shown in the example below. The variable a is initialized with the value 123456, whose binary representation (11110001001000000_2) has 6 digits set to 1. In the first assignment statement, the variable b is assigned with the value of an expression containing `count_ones`. The value of `count_ones` is 6, as its argument is simply the value of a . The rest of the expression adds 100 to this value so, in the end, b is assigned the value 106. In the second assignment, the value of `count_ones` is 13, as the binary representation of the value of the expression $a - 64 + 255$ is 1111000101111111_2 which contains 13 'one' digits. Finally, in the third assignment, the value -1 is represented in binary by 32 'one's, because of two's complement notation. As a result, the value of `count_ones` is 32.

```
int a = 123456, b;

b = count_ones(a) + 100;
/* b == 106 */
b = count_ones(a - 64 + 255);
/* b == 13 */
b = count_ones(-1);
/* b == 32 */
```

1. Define the tokens (and the related declarations in **Acse.lex** and **Acse.y**). (1 points)
2. Define the syntactic rules or the modifications required to the existing ones. (2 points)
3. Define the semantic actions needed to implement the required functionality. (22 points)

4. Given the following Lance code snippet:

$$x[7 + - 3 * y] = z / 10;$$

write down the syntactic tree generated during the parsing with the Bison grammar described in Acse.y *starting from the statement nonterminal*. (5 points)

5. (**Bonus**) Describe how to modify the `count_ones` operator previously implemented to count the bits set to one in the binary representation of the *absolute value* of the argument. In addition, the result of the operator will be positive or negative depending on the sign of the argument. For example, `count_ones(-1)` will have a value of `-1` instead of `32`.